

Sisteme de Operare

Cap 3. Sisteme de fișiere

March 5, 2023

- 1 Interfața sistemului de fișiere
- 2 Structura unui director
- 3 Operații în sistemul de fișiere
- 4 Metode de alocare
- 5 Managementul spațiului liber
- 6 Eficiență și performanțe

- 1 Interfața sistemului de fișiere
- 2 Structura unui director
- 3 Operații în sistemul de fișiere
- 4 Metode de alocare
- 5 Managementul spațiului liber
- 6 Eficiență și performanțe

Noțiunea de fișier

- o colecție de informații căreia i se asociază un nume
- stocat în memoria secundară (memoria primară este RAM-ul)
- informația din fișier este definită de creatorul său; structura depinde de tipul informației stocate în fișier: fișiere text, sursă, executabile

Atributele fișierelor:

- **numele** este informația simbolică asociată conținutului
- **identificatorul** de fișier este un număr cu care este unic asociat în sistemul de fișiere
- **tipul** este necesar pentru asocierarea cu aplicații adecvate
- **locația** e un pointer (adresă) către un dispozitiv și o adresă a informației pe acel dispozitiv
- **mărimea** este dimensiunea curentă în bytes, words, sau blocuri
- **protecția** definește informația de control a accesului, cine poate citi, scrie, executa etc.
- **timestamps** și **user identifier**, informații privind crearea, scrierea, accesul la fișier, respectiv securitatea, protecția și monitorizarea sa

Operații cu fișiere

- **crearea** presupune găsirea de spațiu disponibil, apoi alocarea unei noi intrări în director
- **deschiderea** fișierului presupune găsirea identicatorului din file system asociat cu numele căutat, verificarea permisiunilor și întoarcerea ca rezultat a unei adrese pentru o structură de informații folosite în alte apeluri de sistem pentru fișier
- **scrierea** se face pe baza unui identicator; structura internă ține evidența unui pointer cu locația următoare din filesystem unde se poate scrie informația
- **citirea** se face din fișier în memorie, structura internă va ține evidența următoarei locații de memorie unde se poate scrie, respectiv locația curentă de citire din fișier, aceeași ca la read
- **repoziționarea** (seek) rescrie pointer-ul ce indică poziția curentă din fișier, nu necesită o operație I/O
- **truncate** este ciuntirea fișierului presupune scăderea dimensiunii

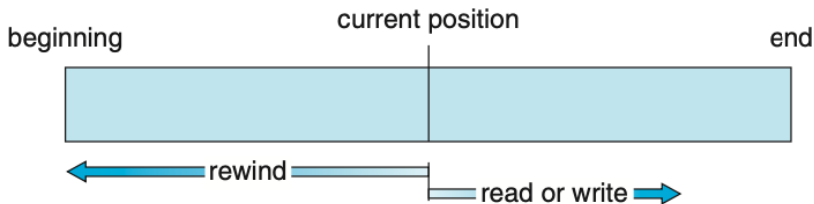
Informații asociate cu fișierul deschis

- **pointer-ul de fișier** este locația din fișier unde se face următoarea citire/scriere; ea va avansa cu dimensiunea blocului scris/citit
- **numărul de deschideri** ale fișierului; SO are o tabelă cu fișierele deschise, și pentru fiecare un counter, fiecare fișier fiind deschis de mai multe procese
- **localizarea fișierului** se referă la informația asociată existenței fișierului, pe filesystem, server NAS sau memorie RAM
- **drepturile de acces** sunt stocate în tabele per-proces, astfel SO poate filtra pe baza sa următoarele apeluri sistem referitoare la fișier
- posibilitatea de a **bloca** (lock) un fișier sau părți din el, pentru acces exclusiv din partea unui proces: shared lock, exclusive lock

Tipuri de fișiere

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter
markup	xml, html, tex	textual data, documents
word processor	xml, rtf, docx	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	gif, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	rar, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, mp3, mp4, avi	binary file containing audio or A/V information

Acces secvențial la fișier



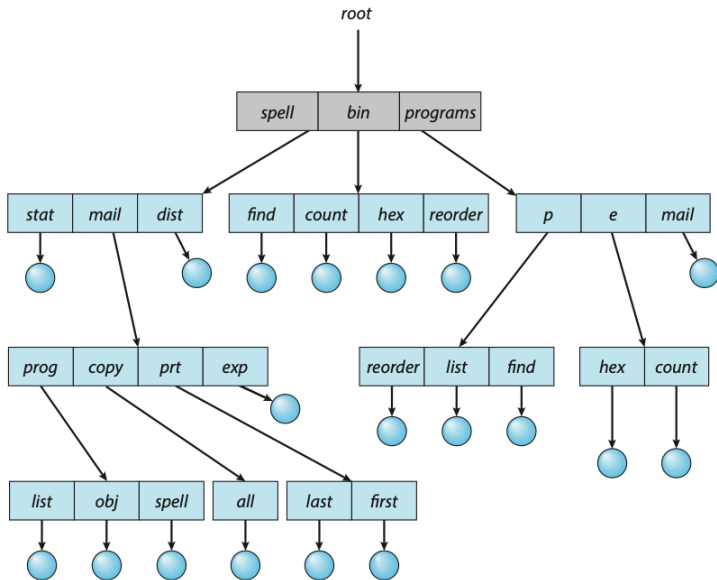
- înregistrările din fișier se citesc secvențial, în ordine
- pointer-ul de acces poate fi resetat la început, sau să 'sară' înainte n înregistrări
- bazat pe modelul de acces secvențial al benzii magnetice
- foarte răspîndit

Acces direct la fișier

- se pot scrie înregistrări rapid, fără o ordine prestabilită
- scrierea/citirea se face pe baza numărului blocului denumit **relative block number**
- un exemplu notabil sunt bazele de date
- simularea accesului direct pe baza unui acces secvențial este puternic ineficient; invers, este relativ simplu, prin păstrarea unei variabile contor

- 1 Interfața sistemului de fișiere
- 2 Structura unui director**
- 3 Operații în sistemul de fișiere
- 4 Metode de alocare
- 5 Managementul spațiului liber
- 6 Eficiență și performanțe

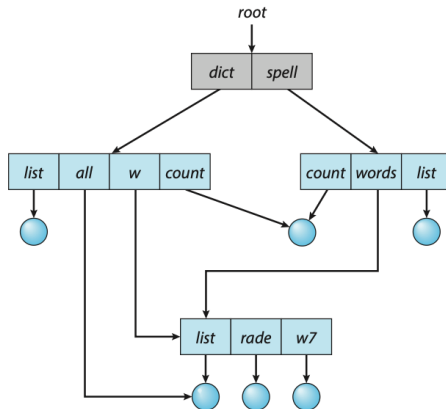
Structura de tip arbore



Noțiuni pentru structura de tip arbore

- un director sau subdirector conține un set de fișiere sau subdirectoare
- **directorul curent** conține cele mai multe fișiere cu care lucrează procesul curent
- **calea absolută** a unui fișier se formează pornind din rădăcină, ex. `/root/bin/hex/a.txt`
- **calea relativă** definește calea unui fișier pornind din directorul curent, ex. calea curentă este `/root/spell`, accesăm fișierul cu `../bin/hex/a.txt`
- pentru ștergerea unui director, unele sisteme impun ca acesta să fie gol; presupune ștergerea tuturor fișierelor; sau, de ex. forțarea ștergerii complete fără confirmare, `rm -rf ..`

Structura de graf aciclic



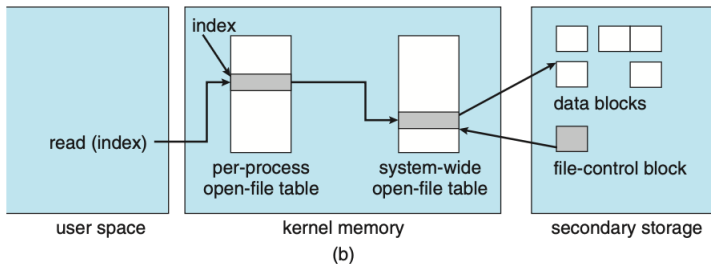
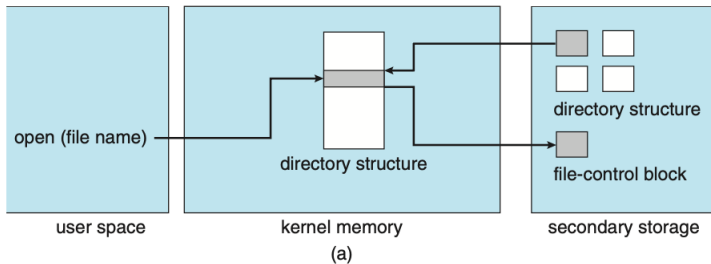
- fișierele partajate se implementează printr-un **link** - un pointer către un alt fișier; conținutul este dat chiar de această cale
- hard links**, fișierul se șterge doar când counter-ul asociat ajunge la 0
- symbolic links**, pointer-ul va indica spre un fișier inexistent.

- 1 Interfața sistemului de fișiere
- 2 Structura unui director
- 3 Operații în sistemul de fișiere**
- 4 Metode de alocare
- 5 Managementul spațiului liber
- 6 Eficiență și performanțe

Structurile sistemului de fișiere

- caching-ul prin RAM a elementelor accelerează operațiile cu fișiere
- informația se încarcă la montarea sistemului de fișiere, folosite și reînnoite la accesul fișierelor, dealocate la demontarea file-system-ului
- **mount table** conține informații despre fiecare volum montat
- fișierele sunt organizate în blocuri; există un **file control block** (FCB) care ține evidența blocurilor ocupate de fișier (adresele lor), drepturile de acces ale fișierului, alte informații asociate (**inode** în Linux)
- system-wide **open file table** conține câte o copie după FCB-ul fiecărui fișier deschis
- per-process open file table conține pointeri către intrările din tabela anterioară pentru fișierele deschise de procesul curent
- citirea/scrierea din/în fișier nu se face direct, ci buffer-at prin memorie, din motive de eficiență (viteză)
- apelul `open()` returnează un pointer în tabela de fișiere, fie **file descriptor** (UNIX) sau **file handle** (Windows)

Apelurile open (a) și read (b)



Implementarea directoarelor ca listă liniară (contiguă)

- directorul este o listă liniară de nume de fișiere cu pointeri către blocurile de date
- consumatoare de timp: la crearea unui nou fișier trebuie verificată toată lista pentru noul nume
- găsirea unui fișier presupune tot parcurgerea întregii liste
- caching-ul celor mai folosite informații din director
- timpul de căutare se poate îmbunătăți dacă lista se menține mereu sortată (aplicăm căutarea binară), dar timpul operațiilor de creare și ștergere a fișierelor crește
- un arbore echilibrat de căutare (red-black tree) poate ajuta performanțele

Directoare implementate cu hash table

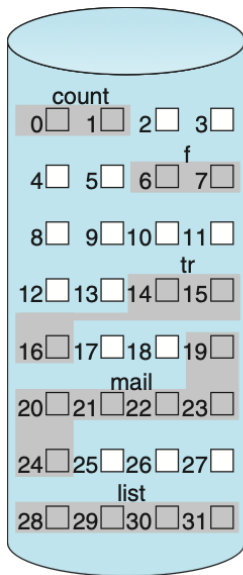
- o listă liniară stochează intrările directoarelor
- tabela de hashing calculează, pe baza numelui de fișier, un pointer din lista liniară care indică spre fișier
- inserția și ștergerea în listă se fac direct, deși trebuie verificat dacă avem coliziuni (două nume de fișier au același hash code, spre aceeași locație)
- dezavantaj major la creșterea dimensiunii tabelii de hashing; ex. toate cele 64 locații s-au umplut, trebuie crescută la 128 locații - recalcularea funcției de hash
- alternativa este folosirea unei liste înlănțuite pentru implementarea fiecărei locații de hash, pentru rezolvarea coliziunilor; performanțe mai bune decât lista liniară simplă

- 1 Interfața sistemului de fișiere
- 2 Structura unui director
- 3 Operații în sistemul de fișiere
- 4 Metode de alocare**
- 5 Managementul spațiului liber
- 6 Eficiență și performanțe

Alocarea contiguă (consecutivă)

- fiecare fișier ocupă un set contiguu de blocuri pe dispozitiv (device)
- alocare potrivită pentru HDD, deoarece accesarea blocului următor nu necesită o mișcare a capului de citire/scriere
- la terminarea cilindrului curent care înmagazinează un număr fix de sectoare, mișcarea spre cilindrul următor este minimă
- alocarea contiguă este definită de adresa primului bloc din fișier și a lungimii sale; dacă startul este la blocul b și ocupă n blocuri, i se vor aloca blocurile $b, b+1, \dots, b+n-1$
- accesul este rapid; atât accesul secvențial cât și cel direct pot fi suportate de alocarea contiguă
- problema alocării constă în găsirea celui mai potrivit bloc de n sloturi, dată fiind o listă de spații libere (free holes); cele mai comune strategii sunt first-fit și best-fit
- algoritmi de alocare suferă de problema **fragmentării externe**

Alocarea contiguă (2)



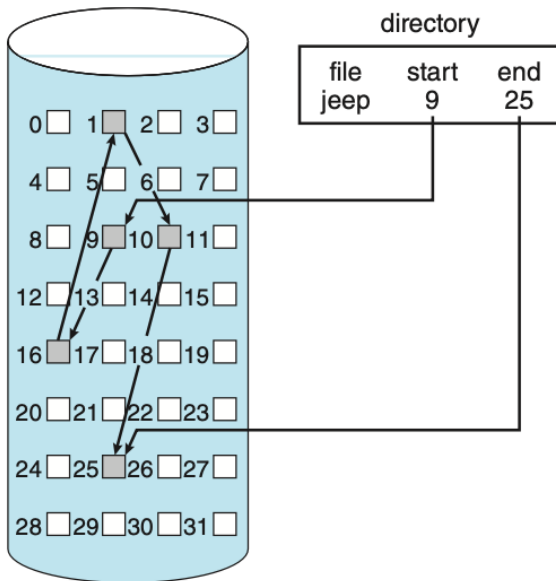
directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Alocarea contiguă (3)

- **fragmentarea internă** se referă la spațiul rămas neutilizat într-un bloc de alocare, pentru că informația fișierului e mai mică decât un multiplu întreg al dimensiunii blocului
- **fragmentarea externă** se referă la fărâmițarea spațiului liber în blocuri din ce în ce mai mici, până când la alocare nu se poate găsi un bloc suficient de larg, deși suma blocurilor libere este mai mare ca lungimea blocului cerut
- o modalitate de **compactare** este copierea întregului sistem de fișiere pe un alt dispozitiv urmată de copierea înapoi, obținându-se o singură zonă liberă - operația trebuie făcută **off-line**, rezultând **down time**
- majoritatea sistemelor moderne pot face compactarea **on-line**, cu penalizarea performanței
- pentru alocarea contiguă, de regulă unui fișier i se alocă mai mult spațiu decât necesar; dacă totuși depășește, fișierul este copiat într-o altă zonă mai mare; fragmentarea este intrinsecă

Alocarea cu liste înlănțuite (1)



Alocarea cu liste înlănțuite (2)

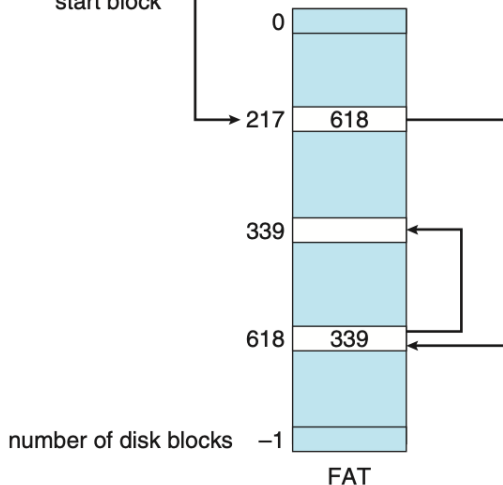
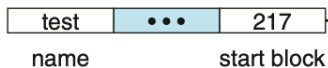
- **alocarea înlănțuită** vede fiecare fișier ca o listă înlănțuită de blocuri; rezolvă problema alocării contigue
- fiecare bloc va conține un pointer către următorul bloc; ex. fiecare bloc are 512 bytes și se folosesc 4 bytes pentru adresa următorului bloc, utilizatorul vede blocuri de 508 bytes
- la alocare, se caută următorul bloc liber, care este adăugat ca bloc ocupat la sfârșitul listei fișierului
- nu există fragmentare externă; citirea se face prin parcurgerea directă a listei
- problema majoră e că se poate folosi doar pentru acces secvențial; accesul direct la blocul i presupune parcurgerea de la început a tuturor blocurilor până ajungem la blocul i
- fiecare acces la pointer presupune o operație de citire a device-ului, pentru un HDD se traduce într-o operație de seek (mutare a capului de citire/scriere, operație consumatoare de timp)

Alocarea cu liste înlănțuite (3)

- o altă problemă este spațiul ocupat cu pointerii - 4 bytes din 512 înseamnă 0.78% din spațiu
- soluția este gruparea blocurilor în multipli, de ex. alocarea în **cluster** de 4 blocuri; se reduce procentul ocupat de pointers dar crește fragmentarea internă
- pot apărea probleme dacă se eronează informația păstrată în pointeri - legături în zona spațiului liber sau în alt fișier; se pot propune liste dublu-înlănțuite - mai mult spațiu utilizat
- un fel de alocare înlănțuită este **File Allocation Table (FAT)**, folosită de MS-DOS:
 - la începutul volumului se păstrează o zonă pentru tabela FAT
 - tabela are o intrare pentru fiecare bloc, indexată după numărul de bloc
 - intrarea în tabelă conține indexul următorului bloc care urmează blocului cu indexul curent
 - valoarea 0 arată că blocul curent este ultimul din fișier
 - tabela FAT se păstrează în cache pentru a reduce numărul de accese la device

Exemplu de File Allocation Table (FAT)

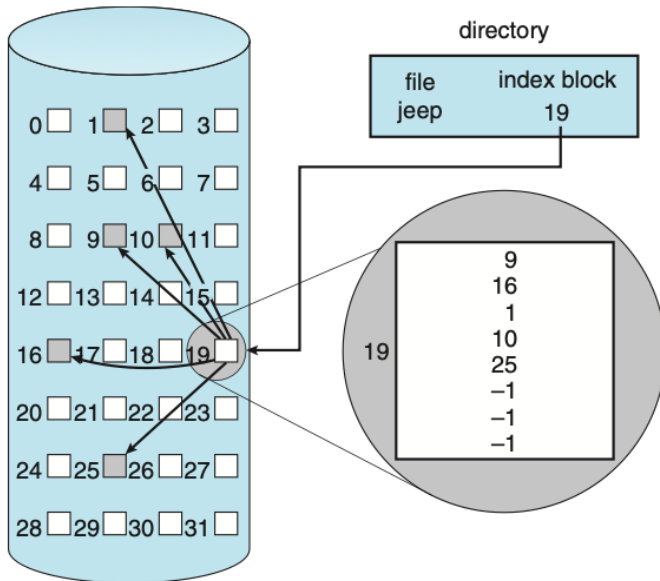
directory entry



Alocarea indexată (1)

- în lipsa tabeli FAT, alocarea înlănțuită nu rezolvă eficient adresarea directă
- alocarea indexată rezolvă alocarea directă prin păstrarea tuturor pointer-ilor în **blocul index**
- fiecare fișier are propriul bloc de index, care este un array de adrese de blocuri
- directorul conține adresa blocului de index
- apare fenomenul de risipă a spațiului, pentru fișiere mici se va alocă un bloc de index chiar dacă doar una-două intrări din acesta vor fi folosite
- dacă index bloc-ul este prea mic, el nu va putea conține suficiente înregistrări să conțină tot fișierul; dacă este prea mare, se face risipă de spațiu, în caz că avem multe fișiere mici

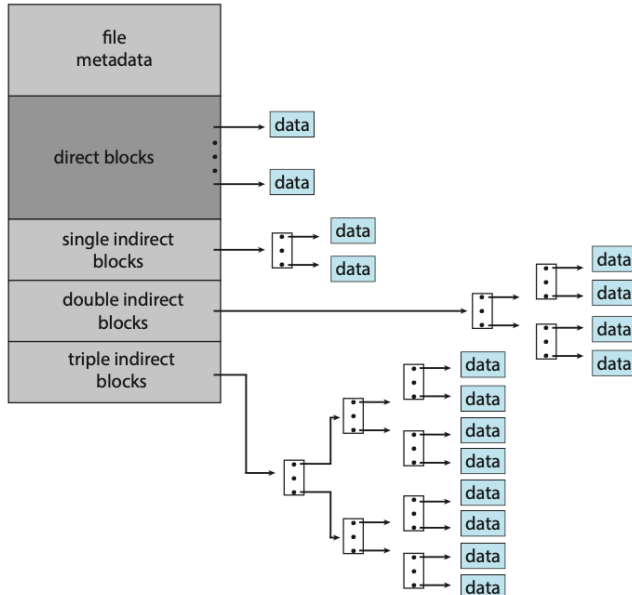
Alocarea indexată (2)



Alocarea indexată (3)

- mecanisme de ajustare a lungimii blocului index:
 - **listă înlănțuită**: un bloc index este privit ca bloc de stocare. Ultima adresă din bloc poate fi NULL pentru un fișier mic, sau un pointer spre un alt bloc index pentru un fișier mare
 - **index multi-nivel**: fiecare bloc index de nivel 1 conține pointeri spre blocuri index de nivel 2, care la rândul lor conțin pointeri spre blocurile de date; ex. pentru blocuri de dimensiune 4096 bytes, putem stoca fișiere de mărime $1024 \times 1024 \times 4096 = 4\text{GB}$
 - **schemă combinată**: sistemele UNIX folosesc de ex. 15 pointeri ai blocului index în inode-ul fișierului; primii 12 indică direct spre blocuri de date; al 13-lea indică spre **single indirect block**, un bloc ce conține doar pointeri spre blocuri de date; al 14-lea este **double indirect block** iar al 15-lea, **triple indirect block**
- de regulă, **index blocks** pot fi cached în memorie, dar blocurile de date pot fi răspândite pe întreg file-system-ul, de aici rezultă aceleași probleme de performanță ca și alocarea înlănțuită

Structura de inode pentru sistemele UNIX

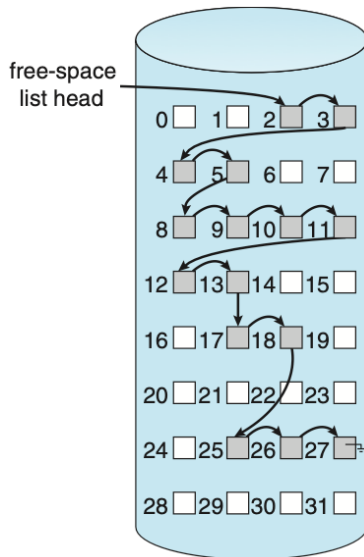


- 1 Interfața sistemului de fișiere
- 2 Structura unui director
- 3 Operații în sistemul de fișiere
- 4 Metode de alocare
- 5 Managementul spațiului liber**
- 6 Eficiență și performanțe

Vector de biți

- evidența blocurilor libere se implementează de regulă cu o listă ce ține evidența blocurilor libere (nealocate); la crearea/adăugarea la un fișier, se caută următorul bloc liber în această listă; nu trebuie implementată neapărat ca listă
- lista de blocuri libere poate fi implementată ca **bitmap**, vector de biți; fiecare bloc liber este marcat ca 0, fiecare bloc alocat ca 1
- metodă simplă și eficientă dacă întregul bitmap este menținut în RAM și scris pe device doar la demontare (unmount)
- un volum de 1 TB (2^{40}) cu blocuri de 4K (2^{12}) va necesita un bitmap de 2^{28} biți = 2^{25} bytes = 32 MB

Liste înlănțuite



- constă în formarea unei liste înlănțuite cu blocurile libere
- lista nu e parcursă frecvent, frecvent se furnizează doar primul bloc
- sistemul FAT încorporează evidența blocurilor libere în aceeași structură

- 1 Interfața sistemului de fișiere
- 2 Structura unui director
- 3 Operații în sistemul de fișiere
- 4 Metode de alocare
- 5 Managementul spațiului liber
- 6 Eficiență și performanțe**

Eficiența

- discurile HDD au performanțe mai reduse decât NVM, iar ambele sunt mai lente decât CPU-ul și memoria principală
- alocarea eficientă a spațiului de stocare depinde de algoritmi folosiți
- în sistemele UNIX, se pre-allocă inode-uri și se răspândesc de-a lungul file system-ului, în ideea de îmbunătăți performanța prin ținerea alăturată a blocului de date de inode-ul fișierului
- ex. fișierele au asociate informațiile de tip "data ultimei scrieri" respectiv "data ultimului acces"; informațiile pot fi actualizate destul de des, astfel încât scrierea FCB-ului la fiecare modificare să fie ineficientă
- pointer-ii la blocurile de date pot utiliza 32 de biți (mărime de fișier de maxim 4GB), respectiv 64 de biți - ceea ce determină mai mult spațiu folosit de structurile de management al spațiului (liste, indecși etc.)

Performanță (1)

- anumite sisteme mențin o secțiune separată a memoriei pe post de **buffer cache**, unde sunt păstrate blocurile aduse de pe device-uri în ideea că ele vor fi utilizate din nou în curând
- alte sisteme pot păstra porțiuni întregi din fișier în **page cache**, ca pagini întregi de memorie, ceea ce este mai eficient decât caching-ul ca blocuri de disc; sisteme precum Linux, Solaris și Windows folosesc mecanismul de page caching pentru a păstra în memorie atât imagini de proces precum și datele din fișiere, cunoscut ca **unified virtual memory**
- un **unified buffer cache** atât pentru device cât și pentru paginile de proces, evită apariția fenomenului de **double caching**: la citirea blocurilor de pe disk, acestea sunt aduse atât în cache cât și în memoria RAM, în spațiul de adrese al procesului; pagina de memorie a procesului este menținută în cache-ul de pagini de proces, deci blocul de date este cache-uit de două ori; se pierde folosesc redundant și timp CPU și memorie

Performanță (2)

- o altă caracteristică ce poate afecta performanța este **scrierea sincronă**; rutina care a comandat scrierea datelor așteaptă ca datele să fie efectiv scrise pe dispozitiv; în **scrierea asincronă**, datele sunt scrise în cache și controlul întors în rutina apelantă; majoritatea scrierilor sunt asincrone
- optimizarea timpului de acces pentru page cache se poate face prin dealocarea ultimei pagini când apare cererea de încărcare a uneia noi - **free-behind**
- o altă tehnică este **read-ahead**, la cererea de citire a unei pagini de program noi, se citesc mai multe, consecutive, în avans, ceea ce salvează semnificativ de mult timp
- la scrierea asincronă a datelor în cantități mici, user-ul percepe timpi foarte mici de scriere, mai mici decât cei de citire; de ex. la scrierea unui fișier mare pe NVM, inițial percepem o viteză mai mare de transfer, care scade în timp

Recovery

- sistemele de fișiere pot deveni inconsistente din cauza defectelor fizice sau întreruperilor abrupte, când informațiile din FCP devin incoerente (scrieri parțiale)
- consistența sistemului de fișiere trebuie verificată la fiecare boot-are; este un proces îndelungat; de regulă, la modificarea metadata-urilor, un bit al sistemului de fișiere (**dirty bit**) este setat; când meta-datele au fost modificate cu succes, bitul este resetat (cleared); la boot-are, dacă acest bit este găsit pe 1 (dirty), se forțează verificarea consistenței sistemului de fișiere
- **consistency checker** verifică coerența informației din metadata cu structura datelor păstrate în directoare

Sisteme de fișiere bazate pe logging

- sistemele de fișiere bazate pe **journaling** se bazează pe menținerea în paralel a unor log-uri (jurnale) a operațiilor I/O efectuate asupra sistemului de fișiere
- pentru Linux, sistemele de fișiere ext3 și ext4 folosesc jurnalizare
- toate schimbările asupra metadatelor sunt scrise și într-un jurnal; system call-ul va returna odată ce jurnalul e scris; intrările în jurnal sunt procesate în ordine de kernel; fiecare set de operații ce îndeplinesc o sarcină alcătuiesc o **tranzacție**
- jurnalul este un **buffer circular**, ce suprascrie cele mai vechi înregistrări
- log-ul este scris într-o zonă separată a fișierului sistemului
- la apariția unui crash, filesystem-ul este adus într-o stare consistentă prin executarea tranzacțiilor neexecutate din jurnal
- scrierea în jurnal se face secvențial, contiguu, este mult mai rapidă decât procesul normal al scrierii inode-urilor împrăștiate în volumul de date

Backup și restore

- dispozitivele de stocare pot înregistra defecte; operația de salvare a datelor pe alt suport se numește **back-up**, iar cea de refacere din datele salvate, **restore**
- de regulă backup-ul curent ține seama de backup-urile anterioare, și nu înregistrează decât modificarea datelor - se economisește spațiu și timp de lucru
 - **backup-ul full** presupune salvarea completă a datelor
 - **backup-ul incremental** presupune doar salvarea diferențelor față de backup-ul realizat anterior (fie el full sau incremental)
- de regulă nu se folosește inițial backup-ul full și apoi doar backup-ul incremental; backup-ul full se face periodic, la un anumit număr de zile
- este o idee bună de a avea backup-urile full stocate fizic în altă locație decât lângă backup-urile incrementale sau calculator (de ex. seif bancar)